AWS Serverless Resume Deployment Guide

RIC Badminton Match Queue
S3 + CloudFront + API Gateway + Lambda + DynamoDB
Generated 2026-06-08

Purpose: deploy your current static badminton queue website as a production-style AWS project that is strong enough to discuss in an Associate Cloud Engineer interview. The guide uses the files already in your folder: **index.html, admin.html, records.html, lambda_function.js**.

Target Resume Bullet

Deployed a serverless web application on AWS using private S3 origin storage, CloudFront CDN with HTTPS and Origin Access Control, API Gateway HTTP APIs, Lambda Node.js backend, DynamoDB persistence, IAM least-privilege access, CloudWatch logs, and repeatable deployment documentation.

Architecture

```
User Browser
-> CloudFront distribution (HTTPS, CDN, default root object)
-> Private S3 bucket (index.html, admin.html, records.html)
-> API Gateway HTTP API (/state)
-> Lambda function (Node.js, admin passcode validation)
-> DynamoDB table (ric_badminton_state)
```

Layer	AWS service	Why it helps your resume
Frontend	Amazon S3	Shows static hosting, object storage, bucket policies, and deployment discipline.
CDN/security	Amazon CloudFront + OAC	Shows HTTPS delivery and a private S3 origin instead of a public bucket.
API	API Gateway HTTP API	Shows managed HTTPS API routing, CORS, stages, and Lambda integration.
Backend	AWS Lambda	Shows serverless compute, environment variables, logs, and IAM execution roles.
Database	Amazon DynamoDB	Shows serverless NoSQL persistence and primary-key design.
Operations	CloudWatch	Shows log-based troubleshooting and basic production monitoring.
Optional polish	Route 53 + ACM	Shows custom domain, DNS, and certificate management.

Before You Start

- Use one AWS Region for Lambda, API Gateway, DynamoDB, and S3. This guide uses us-east-1 as the example.
- Create an AWS account with MFA enabled on the root user. Do daily work from an IAM Identity Center or IAM admin user, not the root user.
- Have your local project files ready: index.html, admin.html, records.html, and lambda_function.js.

- Pick names before starting: S3 bucket name, Lambda function name, API name, DynamoDB table name, and optional domain.

Resource	Recommended value
DynamoDB table	ric_badminton_state
Partition key	key, type String
Lambda function	ric-badminton-api
API Gateway	ric-badminton-gateway
S3 bucket	ric-badminton-static-<your-name-or-id>
CloudFront default root object	index.html
Admin secret variable	ADMIN_PASSCODE

Phase 1: Create DynamoDB

DynamoDB stores the application state. Your Lambda reads and writes several logical records using the partition key named key.

1. Open the AWS Console and search for DynamoDB.
2. Choose Tables, then Create table.
3. Table name: ric_badminton_state.
4. Partition key: key. Type: String.
5. Use On-demand capacity if the console offers billing mode choices. It is practical for low-traffic portfolio projects because you do not need to pre-plan read/write units.
6. Leave encryption at the AWS owned or AWS managed default unless you want to demonstrate KMS separately.
7. Create the table and wait until status is Active.

Why this matters: DynamoDB table design starts with the primary key. AWS documentation describes the partition key as the hash key used to distribute items across partitions.

Phase 2: Create Least-Privilege IAM Policy

Avoid attaching AmazonDynamoDBFullAccess for the final resume version. It works for a lab, but least privilege is a better cloud engineering signal. Create a custom policy that allows this Lambda to read and write only the one table.

In IAM, create a policy similar to this. Replace ACCOUNT_ID and REGION if needed.

```
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": [
"dynamodb:Scan",
"dynamodb:PutItem",
"dynamodb:GetItem",
```

```
"dynamodb:UpdateItem"
],
"Resource": "arn:aws:dynamodb:us-east-1:ACCOUNT_ID:table/ric_badminton_state"
}
]
}
```

Portfolio explanation: the Lambda execution role can access only the application table, not every DynamoDB table in the account.

Phase 3: Deploy Lambda

1. Open Lambda, then choose Create function.
2. Select Author from scratch.
3. Function name: ric-badminton-api.
4. Runtime: choose a supported Node.js runtime available in your account, such as Node.js 20.x or newer.
5. Architecture: x86_64 is fine for this project.
6. Create the function.
7. Open Configuration, Permissions, and click the execution role.
8. Attach your custom DynamoDB policy from Phase 2 to the Lambda execution role.
9. Return to Lambda, open the Code tab, replace the starter code with your local lambda_function.js, then Deploy.

Lambda environment variable

1. Open Configuration, then Environment variables.
2. Add ADMIN_PASSCODE.
3. Set the value to a strong admin passcode.
4. Save and redeploy/test.

Security note: Lambda environment variables are convenient for this project, but do not hard-code secrets into index.html or admin.html. For a more advanced version, move sensitive secrets to AWS Secrets Manager or SSM Parameter Store.

Phase 4: Create API Gateway HTTP API

1. Open API Gateway and choose Create API.
2. Choose HTTP API, then Build.
3. Add integration: Lambda.
4. Select ric-badminton-api.
5. API name: ric-badminton-gateway.
6. Route: ANY /{proxy+}. This lets /state and future endpoints reach the Lambda.
7. Stage: use $default with auto-deploy enabled for the manual version.
8. Create the API and copy the Invoke URL.

Configure CORS

During first setup, you can temporarily allow * as the origin to test quickly. After CloudFront is deployed, tighten the allowed origin to your CloudFront domain or custom domain.

CORS setting	Development value	Final value
Allowed origins	*	https://<cloudfront-domain> or https://<your-domain>
Allowed methods	GET, POST, OPTIONS	GET, POST, OPTIONS
Allowed headers	content-type, x-admin-passcode	content-type, x-admin-passcode

Important: for Lambda proxy-style integrations, your backend response headers still matter. Your Lambda already returns Access-Control-Allow-Origin, Access-Control-Allow-Headers, and Access-Control-Allow-Methods.

Phase 5: Connect Frontend To API

Search index.html, admin.html, and records.html for the API URL or API configuration line. Replace any local or placeholder endpoint with the API Gateway Invoke URL.

```
const API_URL = "https://abc123.execute-api.us-east-1.amazonaws.com";
```

- Public page should call GET /state.
- Admin page should call POST /state and send x-admin-passcode.
- Do not place the real admin passcode visibly in committed frontend code.
- Test in the browser developer console before uploading to S3.

Phase 6: Create Private S3 Bucket

1. Open S3 and choose Create bucket.
2. Bucket name must be globally unique, for example ric-badminton-static-kyaw-2026.
3. Choose the same Region you used for the backend unless you have a reason not to.
4. Keep Block all public access enabled.
5. Keep bucket versioning disabled for the simplest version, or enable it if you want rollback history.
6. Create the bucket.
7. Upload index.html, admin.html, records.html, and any needed CSS/image assets. Do not upload admin secrets.

Do not enable S3 static website hosting for the secure CloudFront OAC design. Use the regular S3 bucket origin, then let CloudFront fetch private objects through OAC.

Phase 7: Create CloudFront Distribution

1. Open CloudFront and choose Create distribution.
2. Origin domain: select the regular S3 bucket origin, not the S3 website endpoint.
3. Origin access: choose Origin access control settings.
4. Create a new OAC with signing behavior set to Sign requests.

5. Viewer protocol policy: Redirect HTTP to HTTPS.
6. Allowed HTTP methods: GET, HEAD is enough for the static site.
7. Default root object: index.html.
8. Create the distribution.
9. Copy the generated S3 bucket policy from CloudFront and paste it into the S3 bucket policy permissions screen.
10. Wait for CloudFront status to deploy, then open the distribution domain name.

> Resume talking point: CloudFront OAC keeps the S3 bucket private. AWS recommends OAC over the older Origin Access Identity approach for modern S3 origins.

Phase 8: Optional Custom Domain

- Register or host the domain in Route 53.
- Request an ACM public certificate in us-east-1 for CloudFront.
- Add the custom domain name as an Alternate domain name in CloudFront.
- Create a Route 53 A/AAAA alias record pointing the domain to the CloudFront distribution.
- Update API Gateway CORS allowed origins to use the custom domain.

Phase 9: Test Plan

Test	Expected result
Open CloudFront URL	index.html loads over HTTPS.
Open /admin.html	Admin UI loads from the same CloudFront distribution.
GET /state through the app	Public data loads from DynamoDB through Lambda.
POST /state without passcode	Returns 401 Unauthorized.
POST /state with passcode	Updates DynamoDB and returns success.
S3 object direct URL	Access denied, because S3 is private.
CloudWatch Lambda logs	Requests and errors are visible for troubleshooting.

Phase 10: Monitoring And Operations

- Open CloudWatch Logs for the Lambda function and verify each test request creates log events.
- Create a CloudWatch alarm for Lambda Errors greater than 0 over 5 minutes.
- Create a second alarm for API Gateway 5XX errors if you want more operations depth.
- Use CloudFront invalidations after uploading new frontend files, for example invalidate /* after major updates.
- Document every AWS resource name in your README so you can explain the architecture later.

Cost Control

Service	Cost control action
S3	Use a small private bucket. Delete unused old assets.
CloudFront	Avoid very large media files. Invalidate only when needed.
Lambda	Keep memory modest, monitor duration and errors.
DynamoDB	Use on-demand for low traffic; avoid unnecessary scans in bigger apps.
API Gateway	HTTP API is usually simpler and lower-cost than REST API for this project.
Route 53	Hosted zones and domains can create monthly charges. Skip custom domain if you need zero recurring DNS cos

Practical advice: set an AWS Budget alert before deploying. Even small projects should have a monthly budget notification.

Common Problems And Fixes

Symptom	Likely cause	Fix
403 from CloudFront	S3 bucket policy or OAC is wrong	Copy the OAC bucket policy again and confirm the distribution ARN.
index.html downloads or shows XML	Wrong origin or object settings	Use regular S3 origin and set CloudFront default root object to index.html.
CORS error in browser	API origin/header/method not allowed	Allow your CloudFront origin and content-type/x-admin-passcode headers
401 from admin save	Missing or incorrect x-admin-passcode	Check browser request headers and Lambda ADMIN_PASSCODE.
500 from API	Lambda runtime/code/IAM issue	Open CloudWatch Logs and inspect the stack trace.
DynamoDB AccessDenied	Lambda role lacks table permission	Attach the custom DynamoDB policy to the execution role.

What To Put On Your Resume

Use one concise project entry. Do not list every console click.

```
Serverless Badminton Match Queue - AWS Cloud Project
- Deployed a static frontend using S3 and CloudFront with private bucket access through
Origin Access Control.
- Built a serverless backend using API Gateway HTTP API, Node.js Lambda, and DynamoDB.
- Implemented server-side admin passcode validation, CORS controls, IAM least-privilege
table access, and CloudWatch logging.
- Documented deployment, testing, troubleshooting, and cost controls for a
production-style portfolio project.
```

Interview explanation:

- Why serverless? Low operations overhead, automatic scaling, and strong fit for a small event or club website.
- Why CloudFront in front of S3? HTTPS, global caching, and secure private-origin access.
- Why API Gateway? It gives a managed HTTPS API boundary in front of Lambda.
- Why DynamoDB? The application needs simple key-value state persistence without managing servers.

- What would you improve next? Infrastructure as code with SAM, CloudFormation, Terraform, or CDK; CI/CD; custom domain; better auth.

Future Upgrade Path

1. Infrastructure as code: define S3, CloudFront, API Gateway, Lambda, DynamoDB, IAM, and alarms in Terraform, CDK, SAM, or CloudFormation.
2. CI/CD: GitHub Actions uploads static files to S3, deploys Lambda, and creates CloudFront invalidations.
3. Authentication: replace shared passcode with Cognito user authentication for a stronger production design.
4. Data model: replace table scans with direct GetItem calls when the state keys are known.
5. Observability: add structured Lambda logs and CloudWatch dashboard widgets.

Official AWS References

- CloudFront OAC for S3 origins: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
- CloudFront standard distribution with S3 origin: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/GettingStarted.SimpleDistribution.html
- API Gateway HTTP APIs: https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api.html
- API Gateway CORS behavior: https://docs.aws.amazon.com/apigateway/latest/developerguide/how-to-cors.html
- Lambda environment variables: https://docs.aws.amazon.com/lambda/latest/dg/configuration-envvars.html
- DynamoDB table basics: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/WorkingWithTables.Basics.html

Email Draft

To: kyawhtethein.tp@gmail.com

Subject: AWS serverless deployment guide for my resume project

Hi,

Attached is the AWS serverless deployment guide for my RIC Badminton Match Queue portfolio project. It explains the architecture, step-by-step AWS setup, security decisions, testing plan, troubleshooting notes, and resume bullets.

The deployment uses S3, CloudFront, API Gateway, Lambda, DynamoDB, IAM, and CloudWatch so the project demonstrates practical AWS Associate Cloud Engineer skills.

Best,
Kyaw